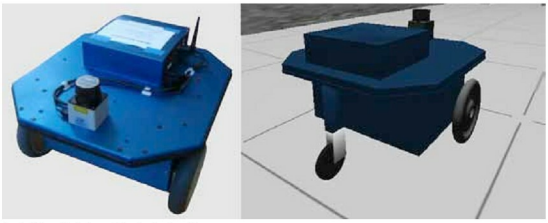
Open Source Robotics
ROS Revisited



Part—3

Erratic Robot : Real robot and the simulated model

The third article in this series focuses on open source software for robotics. In Part 1, the reader was introduced to ROS (Robot Operating System). Here, a few more features of ROS are discussed and a novel aspect of mobile robots is introduced—Simultaneous Localisation and Mapping (SLAM).

We are now ready to delve deeper into some more interesting features of the Robot Operating System (ROS). Let's explore the basic architecture of ROS, teleoperation, the graphical tool rviz, SLAM, and a GUI for ROS.

These discussions are derived from basic examples and default tutorials for ROS. I hope that avid Linux enthusiasts will make their way to *http://www.ros.org/wiki*, *http://answers.ros.org* and *http://planet.ros.org/*.

Basic architecture of ROS

ROS works via a client-server model, where the server is *roscore*. Starting *roscore* is often the first thing to do in order to work with ROS. Tools such as *roscd*, *rxgraph*, *roswtf*, etc, are provided for easy navigation in the ROS files and for debugging. The next section will illustrate the use of these commands, interfaces and tools.

To really enjoy the next few sections it is advisable to install ROS Electric. An Ubuntu 10.04 LTS installation has been used to test the simulations discussed here. For more information on ROS installation, visit *http://www.ros.org/wiki/ROS/Installation*.

Also, some packages need to be built with *rosmake* before we start the next section:

```
rosmake stage
rosmake teleop_base
rosmake gmapping
```

Teleoperation

Teleoperation is to control an actual or simulated robot, using real-time user input. Here, we discuss the erratic robot in *stage* simulation; the user input is via keyboard. To orchestrate teleoperation, it is necessary to invoke the master, as follows:

```
roscore
```

Figure 1: rviz, a visualisation tool in ROS